

Conclusion

This experiment demonstrated the installation and practical usage of Docker for container-based application deployment. Docker provides a lightweight, efficient, and portable environment for running applications, making it highly suitable for modern DevOps and cloud-native architectures.

Viva-Voce Questions

- 1.What is a Docker Image?
- 2.What is a Docker Container?
- 3.What is the difference between docker run and docker start?
- 4.What is the purpose of port mapping in Docker?
- 5.Why are containers lightweight compared to Virtual Machines?

Overall Lab Conclusion

This lab demonstrated virtualization using Vagrant and VirtualBox and containerization using Docker. The experiment clearly showed the performance and resource efficiency advantages of containers. Containers are better suited for rapid deployment and microservices, whereas Virtual Machines provide stronger isolation.

EXPERIMENT – 03

Aim

To deploy the NGINX web server using different Docker base images and compare their image size, performance, layers, and security impact.

Objectives

After completing this experiment, students will be able to:

- Deploy NGINX using Docker containers
- Build custom images using Dockerfile
- Understand Docker image layers
- Compare Ubuntu, Alpine, and Official images
- Analyze size, speed, and security differences
- Explain real-world uses of NGINX in containers

Prerequisites

- Docker installed and running
- Basic Linux commands
- Knowledge of:
 - `docker pull`
 - `docker run`
 - `docker build`
 - Dockerfile
 - Port mapping

Theor y

What is NGINX?

NGINX is a high-performance:

- Web Server
-
-

Reverse Proxy
Load Balancer
API Gateway

It uses an event-driven asynchronous architecture, making it faster and more scalable than traditional servers like Apache.

What is Docker?

Docker is a containerization platform that:

- Packages applications with dependencies
- Ensures portability
- Provides lightweight virtualization
- Speeds up deployment

What are Docker Image Layers?

Docker images are built in layers.

Each instruction in a Dockerfile creates a layer:

```
FROM  
RUN  
COPY  
ADD
```

Importance of Layers

- More layers → Bigger image
- Bigger images → Slower pull time
- Larger images → More vulnerabilities
- Fewer layers → Faster and more secure

Base Image Types

Base Image	Description
------------	-------------

Official nginx Pre-built optimized production image

Ubuntu Full Linux OS with tools

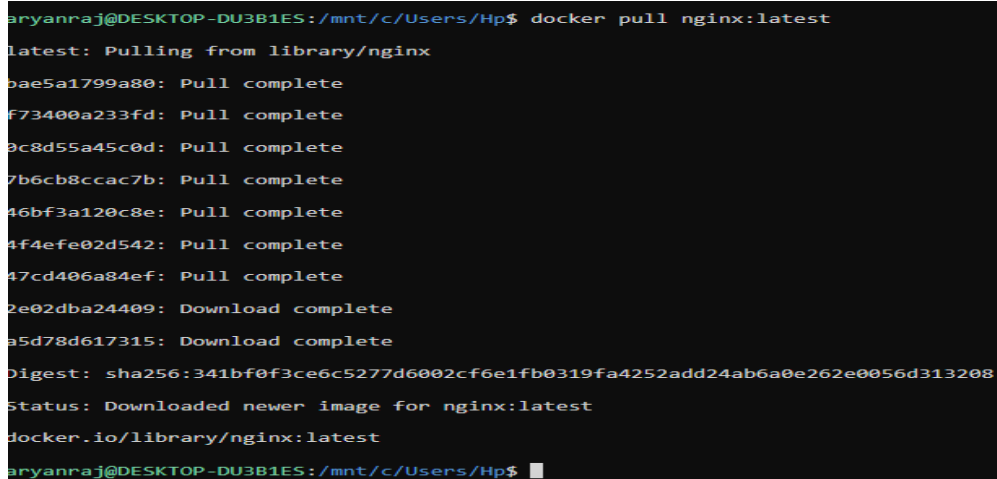
Alpine Lightweight minimal Linux

Experiment Procedure

Part 1 — Deploy NGINX Using Official Image

Step 1: Pull Image

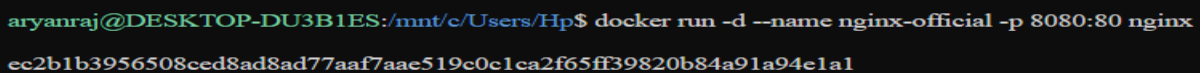
```
docker pull nginx:latest
```



```
aryanraj@DESKTOP-DU3B1ES:/mnt/c/Users/Hp$ docker pull nginx:latest
latest: Pulling from library/nginx
bae5a1799a80: Pull complete
f73400a233fd: Pull complete
9c8d55a45c0d: Pull complete
7b6cb8ccac7b: Pull complete
46bf3a120c8e: Pull complete
4f4efe02d542: Pull complete
47cd406a84ef: Pull complete
2e02dba24409: Download complete
a5d78d617315: Download complete
Digest: sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
aryanraj@DESKTOP-DU3B1ES:/mnt/c/Users/Hp$
```

Step 2: Run Container

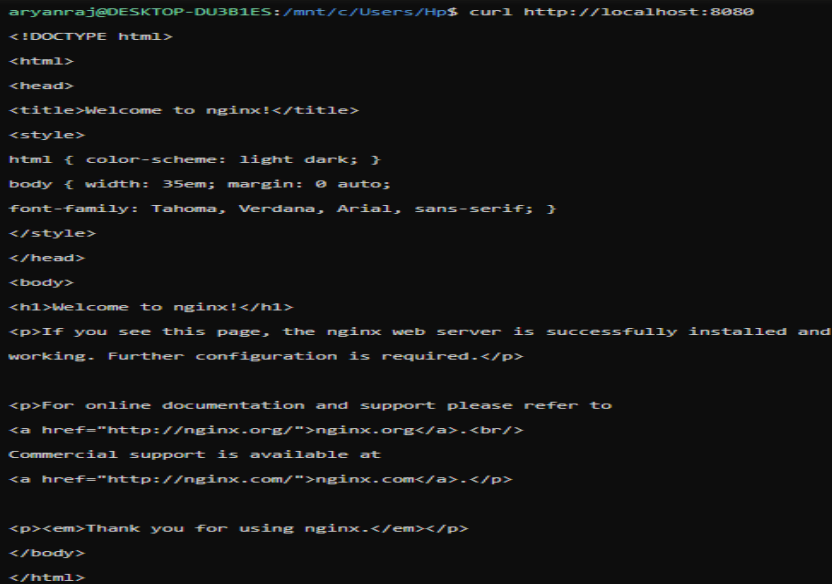
```
docker run -d --name nginx-official -p 8080:80 nginx
```



```
aryanraj@DESKTOP-DU3B1ES:/mnt/c/Users/Hp$ docker run -d --name nginx-official -p 8080:80 nginx
ec2b1b3956508ced8ad8ad77aaf7aae519c0c1ca2f65ff39820b84a91a94e1a1
```

Step 3: Verify

```
curl http://localhost:8080
```

A terminal window with a black background and green text. The prompt is 'aryanraj@DESKTOP-DU3B1ES:/mnt/c/Users/Hp\$'. The command 'curl http://localhost:8080' has been executed, resulting in an HTML response from an nginx server. The HTML includes a title 'Welcome to nginx!', a style block with a light dark color scheme and Tahoma font, and a body with a welcome message and links to nginx.org and nginx.com.

```
aryanraj@DESKTOP-DU3B1ES:/mnt/c/Users/Hp$ curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

OR open in browser:

```
http://localhost:8080
```

Observations

```
docker images nginx
```

- Pre-optimized
- Minimal configuration
- Production ready
- Medium size (~140MB)

Part 2 — Custom NGINX Using Ubuntu Base Image

Step 1: Create Dockerfile

Create a file named Dockerfile :

```
FROM ubuntu:22.04
```

```
RUN apt-get update && \ apt-  
get install -y nginx && \  
apt-get clean && \ rm -rf  
/var/lib/apt/lists/*
```

```
EXPOSE 80
```

```
CMD ["nginx", "-g", "daemon off;"]
```

Step 2: Build Image

```
docker build -t nginx-ubuntu .
```

```
aryanraj@DESKTOP-DU381ES:/mnt/c/Users/Hp$ docker build -t nginx-ubuntu  
ERROR: docker: 'docker buildx build' requires 1 argument  
  
Usage:  docker buildx build [OPTIONS] PATH | URL | -  
  
Run 'docker buildx build --help' for more information  
aryanraj@DESKTOP-DU381ES:/mnt/c/Users/Hp$
```

Step 3: Run Container

```
docker run -d --name nginx-ubuntu -p 8081:80 nginx-ubuntu
```

```
=> => unpacking to docker.io/library/nginx-ubuntu:latest 0.8s  
PS C:\Users\HP\Desktop> docker run -d --name nginx-ubuntu -p 8081:80 nginx-ubuntu  
b02cba283b5d482e7136c97413f9cba2a215158d928979b67a5156749ffe3786  
PS C:\Users\HP\Desktop> @  
>> FROM alpine:latest  
>> RUN apk add --no-cache nginx  
>> EXPOSE 80  
>> CMD ["nginx", "-g", "daemon off;"]  
>> "@ | Set-Content Dockerfile -Encoding UTF8  
PS C:\Users\HP\Desktop> docker build -t nginx-alpine .  
[+] Building 71.6s (7/7) FINISHED docker:desktop-linux
```

Observations

```
docker images nginx-ubuntu
```

- Large image (~220MB+)
- Many layers
- Includes full OS
- Slower startup
- Larger attack surface

```
=> => unpacking to docker.io/library/nginx-alpine:latest 0.1s
PS C:\Users\HP\Desktop> docker images nginx-alpine
IMAGE                ID                DISK USAGE  CONTENT SIZE  EXTRA
nginx-alpine:latest  5a244f3a347e      16MB        4.74MB
PS C:\Users\HP\Desktop> docker images nginx-ubuntu
IMAGE                ID                DISK USAGE  CONTENT SIZE  EXTRA
nginx-ubuntu:latest  9aebf0344388      199MB       52.7MB
```

Part 3 — Custom NGINX Using Alpine Base Image

Step 1: Create Dockerfile

```
FROM alpine:latest

RUN apk add --no-cache nginx

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Step 2: Build Image

```
docker build -t nginx-alpine .
```

```

PS C:\Users\HP\Desktop> docker build -t nginx-alpine .
[+] Building 71.6s (7/7) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 134B                               0.0s
=> [internal] load metadata for docker.io/library/alpine:latest    14.9s
=> [auth] library/alpine:pull token for registry-1.docker.io       0.0s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 2B                                     0.0s
=> [1/2] FROM docker.io/library/alpine:latest@sha256:25109184c71bda 14.7s
=> => resolve docker.io/library/alpine:latest@sha256:25109184c71bdad 0.0s
=> => sha256:589002ba0eaed121aldbf42f6648f29e5be55d 3.86MB / 3.86MB 14.3s
=> => extracting sha256:589002ba0eaed121aldbf42f6648f29e5be55d5c8a6e 0.3s
=> [2/2] RUN apk add --no-cache nginx                             40.5s
=> exporting to image                                              0.5s
=> => exporting layers                                           0.3s
=> => exporting manifest sha256:e2db2ad75194e8ab945318a6255f4da74df0 0.0s
=> => exporting config sha256:6f42d3f9b087816f9afeaecbfeea139c68f1a5 0.0s
=> => exporting attestation manifest sha256:a7a12ae6c26ecd9d4b732ed1 0.1s
=> => exporting manifest list sha256:5a244f3a347e9b77c36be2a605923f3 0.0s
=> => naming to docker.io/library/nginx-alpine:latest            0.0s
=> => unpacking to docker.io/library/nginx-alpine:latest         0.1s
PS C:\Users\HP\Desktop> docker images nginx-alpine

```

Step 3: Run Container

```
docker run -d --name nginx-alpine -p 8082:80 nginx-alpine
```

```

PS C:\Users\HP\Desktop> docker run -d --name nginx-alpine -p 8082:80 nginx-a
lpine
c2ead83e7e7ee71b5d6a853f95a56fe3da51fed6638f6996df8e7ef95f04a37e
PS C:\Users\HP\Desktop> curl http://localhost:8080

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the
web page might be run when the page is parsed.
RECOMMENDED ACTION:
Use the -UseBasicParsing switch to avoid script code execution.

Do you want to continue?

[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help
(default is "N"):Y

```

Observations

```
docker images nginx-alpine
```

- Very small (~25-30MB)
- Minimal dependencies
- Faster pull time
- Faster startup
- More secure

Part 4 — Compare Image Sizes

```
docker images | grep nginx
```

```
PS C:\Users\HP\Desktop> docker images | Select-String "nginx"
WARNING: This output is designed for human readability. For machine-readable
output, please use --format.

nginx-alpine:latest      5a244f3a347e      16MB      4.74MB      U
nginx-ubuntu:latest     9aebf0344300      199MB      52.7MB      U
nginx:latest             9dd288848f44      240MB      65.7MB      U
```

Sample Comparison

Image Type Size nginx:latest

~140MB nginx-ubuntu

~220MB+ Image Type Size

nginx-alpine ~25MB

Part 5 — Inspect Image Layers

```
docker history nginx
docker history nginx-ubuntu
ubuntu docker history
nginx-alpine
```

```
docker history nginx
docker history nginx-ubuntu
docker history nginx-alpine
```

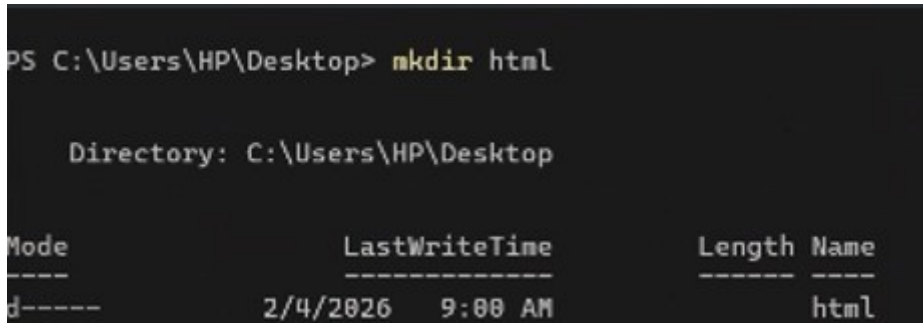
Observations

- Ubuntu → Many filesystem layers
- Alpine → Minimal layers
- Official → Optimized layers

Part 6 — Serve Custom HTML Page

Step 1: Create HTML File

```
mkdir html
echo "<h1>Hello from Docker NGINX</h1>" > html/index.html
```



Step 2: Run Container with Volume Mapping

```
docker run -d \
  -p 8083:80 \
  -v $(pwd)/html:/usr/share/nginx/html \
  nginx
```

Step 3: Verify

Open in browser:

<http://localhost:8083>

```

PS C:\Users\HP\Desktop> Set-Content -Path "html\index.html" -Value "<h1>Hello from Docker NGINX - My Custom Page</h1>"
PS C:\Users\HP\Desktop> docker run -d --name nginx-custom -p 8083:80 -v ${PWD}\html:/usr/share/nginx/html nginx
2658f8b6c5fab5186e32f01604ef1c8c1b52dc246ef7e2981fbc2bb199304cd6
PS C:\Users\HP\Desktop> curl http://localhost:8083

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the
web page might be run when the page is parsed.
RECOMMENDED ACTION:
Use the -UseBasicParsing switch to avoid script code execution.

Do you want to continue?

[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help
(default is "N"):Y

StatusCode      : 200
StatusDescription : OK
Content         : <h1>Hello from Docker NGINX - My Custom Page</h1>

```

Part 7 — Real World Uses of NGINX

NGINX is commonly used for:

- Static websites
- Reverse proxy
- Load balancing
- SSL termination
- API gateway
- Kubernetes ingress controller
- Microservices frontend

Result

NGINX was successfully deployed using three different base images. The size, performance, and security impact of each base image were analyzed and compared.

Conclusion

This experiment demonstrated how base images significantly impact Docker image size, performance, and security.

- Official image → Optimized and production-ready
- Ubuntu → Large and feature-rich but
- heavier Alpine → Lightweight, fast, and secure

Alpine-based images are preferred for lightweight production deployments, while Ubuntu may be used when additional tools or libraries are required.